

Library Information System

Java Programming, Project 2

Jules Christophe Ludovic Roulleau

University of Split, FESB

Course: Programming in Java (FELP11)

Instructor: Assoc. Prof. Vladimir Pleština, PhD

Academic year: 2025 / 2026

Github: <https://github.com/julestek/java-project-roulleau-2026>

1. Introduction

This report describes the Java desktop application I built for the second project of the Programming in Java course. The application is a Library Information System. It simulates a library catalogue and a lending system, where library staff can manage a collection of items, register members, lend and return items, and search the catalogue through a graphical interface.

The application handles four types of library items organised in an inheritance hierarchy: books, magazines, DVDs and audiobooks. A member can borrow an item if a copy is available, and each loan is linked to the member through a loan record. Every loan has a borrow date and a due date set 14 days later. When an item is returned late, the application calculates a fine of 0.20 EUR for each day of delay.

All the data is saved into plain CSV files inside a data folder. The catalogue, the members and the loans are saved automatically when I close the application, and they are loaded again automatically when it starts. This way nothing is lost between two runs, and the overdue calculation is still correct after a restart because it is recomputed from the borrow date and the current date.

The main design choices I made were the following. First, I created one abstract class called `LibraryItem` that holds the fields and the behaviour shared by all item types. Second, I defined two interfaces, `Borrowable` and `Searchable`, to describe what an item can do, separately from its concrete type. Third, I kept the graphical interface completely separate from the logic, so the GUI only calls methods on the `Library` class and never touches the data structures directly. I explain these choices in more detail in sections 4 and 5.

1.1 Scope and limitations

Because of the limited time available, I chose to prioritise the parts of the project that carry the most marks and that show the core Java concepts of the course: the full class hierarchy, the custom exceptions, the collections, the CSV persistence and a working three-tab graphical interface. As a result, two optional features described in the assignment were left out for now. I could add them later, but I preferred to send a first version that works well without them rather than a version where everything is only half finished. The two features I did not implement are:

- The dynamic Add Item dialog with a `CardLayout` that changes the form fields depending on the selected item type. For now the items come from the CSV data files, which act as the demonstration catalogue. Adding, finding and removing items is already fully supported inside the `Library` class, only the input form is missing.
- The bonus background thread that checks for overdue loans every 60 seconds and updates a badge on the tab. The overdue detection itself works, it is just recalculated every time I refresh the Loans tab instead of automatically by a thread.

2. Application features

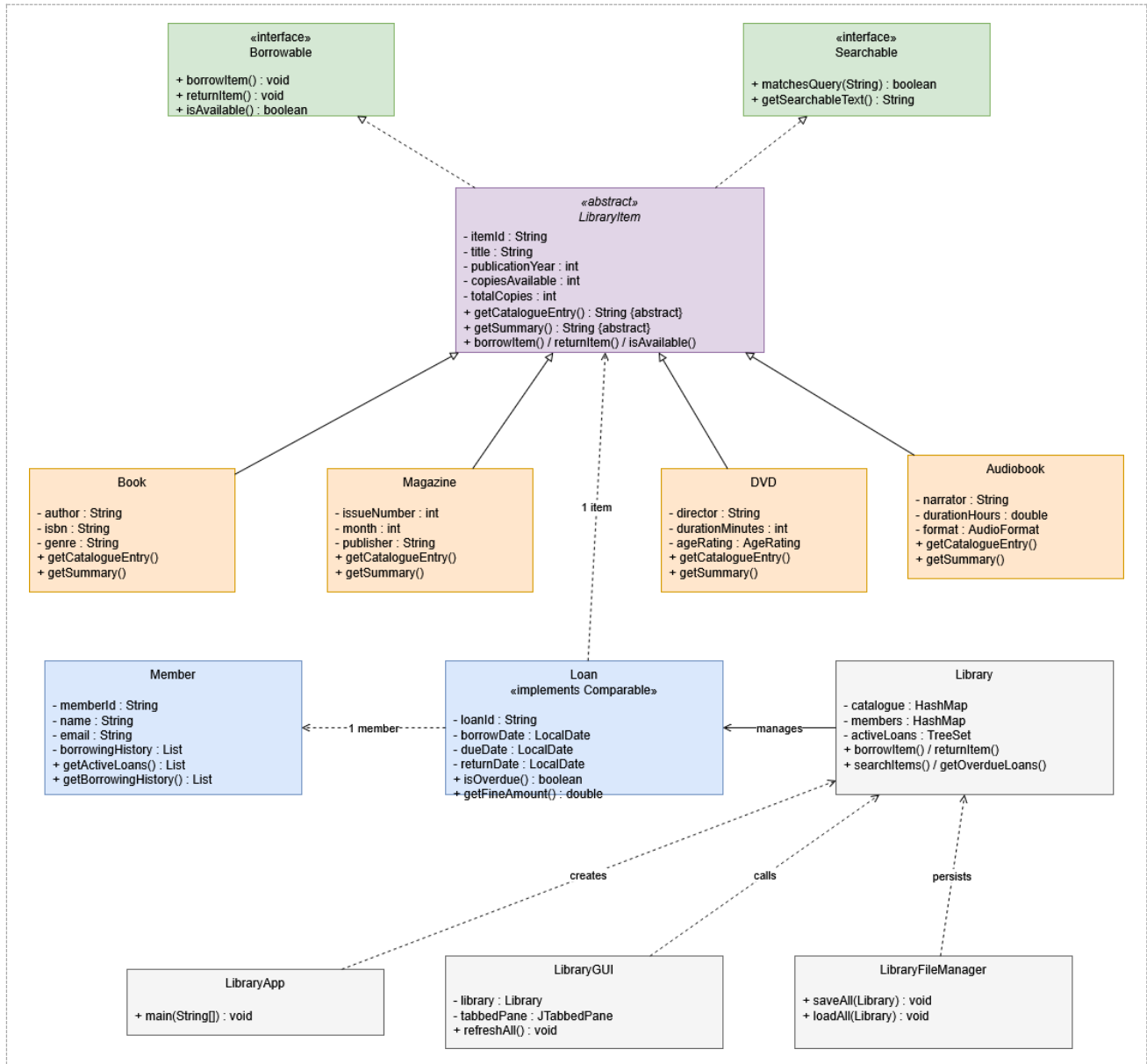
The application offers the following operations:

1. Manage the catalogue: add, find and remove items, with validation of the required fields. For example a book ISBN must be exactly 13 digits, and a publication year cannot be in the future.
2. Register members and view a member profile with the active loans and the full borrowing history.
3. Borrow an item: link a member to an available copy and set the due date to the borrow date plus 14 days.
4. Return an item: update the availability and, if the item is late, calculate and show the fine at 0.20 EUR per day.
5. Search and filter the catalogue by text (title, author, director, narrator), by item type, and by availability.
6. View the borrowing history of a selected member.
7. Show the active loans, with the overdue ones highlighted in red and their fine displayed.

All the items, members and loans are saved to CSV files and loaded again when the application starts.

3. UML class diagram

The diagram below shows the full class structure: the two interfaces, the abstract class, its four subclasses, and the other classes Member, Loan, Library, LibraryFileManager and LibraryGUI.



In the diagram, the dashed arrows from **LibraryItem** to the two interfaces mean that the abstract class implements both interfaces (realisation). The plain arrows with a hollow triangle, going from the four subclasses to **LibraryItem**, mean inheritance. The other connectors show the associations between **Loan**, **Member** and **LibraryItem**, and between **Library** and the rest of the classes.

4. Implementation details

4.1 Package structure

All the classes use the package prefix `hr.fesb.java.library`, which follows the usual reversed-domain Java naming convention. I split the code into four sub-packages so that each one has a clear job:

- `model`: the domain classes, which are `LibraryItem` and its subclasses, the two interfaces, `Member`, `Loan` and `Library`.
- `exceptions`: the three custom exception classes.
- `io`: the `LibraryFileManager` class, which does all the CSV reading and writing.
- `gui`: the Swing interface, which is `LibraryGUI` and the three tab panels.

4.2 The interface hierarchy

Two interfaces describe what a catalogue item can do. `Borrowable` declares the operations needed to lend an item (`borrowItem`, `returnItem` and `isAvailable`). `Searchable` declares the operations needed to search it (`matchesQuery` and `getSearchableText`). The abstract class `LibraryItem` implements both of them.

I chose to use two interfaces instead of putting every method directly on `LibraryItem` because they describe two different roles. One role is about lending, the other is about searching. Since `LibraryItem` implements both, every concrete item automatically gets the two roles, but in the design they stay separate, which I find cleaner and easier to extend later.

4.3 The abstract base class

`LibraryItem` is declared abstract, so it cannot be created directly. It holds the fields that all items share (`itemId`, `title`, `publicationYear`, `copiesAvailable`, `totalCopies`). They are all private and are accessed through getters and setters. The constructor checks every argument and throws an `IllegalArgumentException` for an empty title, an invalid year or a number of copies that is not positive.

The class also contains the borrowing logic that is the same for every item, and it declares two abstract methods, `getCatalogueEntry` and `getSummary`, that each subclass must write differently. This makes sure that each item type produces its own CSV line and its own readable description, while still reusing all the common code.

4.4 The four subclasses

`Book`, `Magazine`, `DVD` and `Audiobook` each extend `LibraryItem`. They add their own fields with validation and override the abstract methods and `toString`. For example, `Book` adds an author, a 13-digit ISBN and a genre, and checks the ISBN format in its constructor. `DVD` and `Audiobook` use enum types (`AgeRating` and `AudioFormat`) for their classification fields, which means only valid values can ever be stored.

4.5 The Library class and the GUI separation

Library is the main logic class. It stores the items in a HashMap with the item ID as the key, the members in a HashMap with the member ID as the key, the active loans in a TreeSet sorted by due date, and the finished loans in a list. It offers methods such as addItem, registerMember, borrowItem, returnItem, searchItems and getOverdueLoans.

The graphical interface never reads these collections directly. Each panel keeps a reference to the Library object and calls its public methods, then shows the result. Keeping the interface and the logic separate was one of the goals of the project, and it made the table models much easier to write.

5. Java concepts applied

In this section I link each Java concept from the assignment to the place where I used it, with a short code example.

5.1 OOP, classes and objects

The project has well over six non-trivial classes. All the fields are private and accessed through getters and setters, and the constructors validate their input.

```
public LibraryItem(String itemId, String title, int publicationYear, int totalCopies) {
    if (title == null || title.trim().isEmpty())
        throw new IllegalArgumentException("Title cannot be empty.");
    if (publicationYear < 1 || publicationYear > LocalDate.now().getYear())
        throw new IllegalArgumentException("Invalid publication year.");
    this.title = title.trim();
}
```

5.2 Inheritance and polymorphism

The abstract class `LibraryItem` has four subclasses. Each one overrides `getSummary` to return its own description. This is polymorphism: the same call gives a different result depending on the real type of the object.

```
@Override
public String getSummary() {
    return String.format("\"%s\" by %s, %s (%d)",
        getTitle(), author, genre, getPublicationYear());
}
```

5.3 Abstract classes

`LibraryItem` is abstract and has the abstract method `getCatalogueEntry`, which every subclass must implement to produce its own CSV line.

```
public abstract class LibraryItem implements Borrowable, Searchable {
    public abstract String getCatalogueEntry();
    public abstract String getSummary();
}
```

5.4 Interfaces

I defined two interfaces, `Borrowable` and `Searchable`. They are implemented by `LibraryItem`, so all the item types implement them too.

```
public interface Borrowable {
    void borrowItem() throws ItemNotAvailableException;
    void returnItem();
    boolean isAvailable();
}
```

5.5 Exception handling

I created three custom exceptions. `InvalidISBNException` is unchecked (it extends `RuntimeException`) because it signals a data error found when an object is built. `ItemNotAvailableException` and `MemberNotFoundException` are checked (they extend

Exception) because the caller is supposed to handle them, for example by showing an error dialog in the GUI.

```
try {
    Loan loan = library.borrowItem(memberId, itemId);
} catch (ItemNotAvailableException | MemberNotFoundException e) {
    JOptionPane.showMessageDialog(this, e.getMessage(),
        "Error", JOptionPane.ERROR_MESSAGE);
}
```

5.6 Collections and sorting

I used ArrayList for the result lists, HashMap to index the items and the members by their ID, and a TreeSet to keep the loans sorted by due date. Loan implements Comparable so the TreeSet can sort the loans on its own, and I used a Comparator to sort the search results by title.

```
@Override
public int compareTo(Loan other) {
    int cmp = this.dueDate.compareTo(other.dueDate);
    return cmp != 0 ? cmp : this loanId.compareTo(other.loanId);
}

results.sort(Comparator.comparing(LibraryItem::getTitle,
    String.CASE_INSENSITIVE_ORDER));
```

5.7 File I/O

The LibraryFileManager class reads and writes all the data with BufferedReader and BufferedWriter, and it handles IOException so that a missing or unreadable file never crashes the application.

```
try (BufferedWriter bw = new BufferedWriter(new FileWriter(ITEMS_FILE, false))) {
    for (LibraryItem item : library.getAllItems()) {
        bw.write(item.getCatalogueEntry());
        bw.newLine();
    }
} catch (IOException e) {
    System.err.println("Error saving items: " + e.getMessage());
}
```

5.8 Helper classes (java.time)

The due dates, the return dates and the overdue calculation use java.time.LocalDate and ChronoUnit. I used String.format to show the fine with two decimals.

```
public long getDaysOverdue() {
    if (!isOverdue()) return 0;
    return ChronoUnit.DAYS.between(dueDate, LocalDate.now());
}

public double getFineAmount() {
    return getDaysOverdue() * 0.20;
}
```

5.9 GUI, Swing

I built the interface with Swing. It uses a JFrame with a JTabbedPane that has three tabs, a JTable with my own AbstractTableModel subclasses for the catalogue and the loans, a JTextField and a

JComboBox for the search bar, plus buttons and labels. The overdue rows in the loans table are coloured in red by overriding prepareRenderer.

```
public Component prepareRenderer(TableCellRenderer renderer, int row, int col) {  
    Component c = super.prepareRenderer(renderer, row, col);  
    Loan loan = tableModel.getLoanAt(row);  
    c.setBackground(loan.isOverdue() ? new Color(255, 180, 180) : Color.WHITE);  
    return c;  
}
```

6. Testing

I tested the application in two steps. First, I tested all the logic from the command line, before I had any graphical interface. I wrote a small test routine in the main method that used each class and printed the result of each case, including the validation errors and the custom exceptions. This let me check that the model worked correctly on its own.

At the logic level I checked these cases:

- Constructor validation: empty titles, invalid years, wrong copy counts, bad ISBNs and out-of-range months were all rejected with the right exception.
- Borrowing logic: borrowing reduced the availability, borrowing an item with no copy left threw `ItemNotAvailableException`, and returning an item restored the availability.
- Overdue and fine calculation: a loan with a borrow date 20 days in the past was correctly reported as overdue, with the right number of days and the right fine.
- Search and filter: searching by text, by type and by availability returned the right items.
- Persistence: after saving and loading, the number of items, members and loans matched, and the overdue calculation was still correct after a restart.

Then, once the interface was ready, I tested each tab and dialog by hand. The screenshots below show the running application.

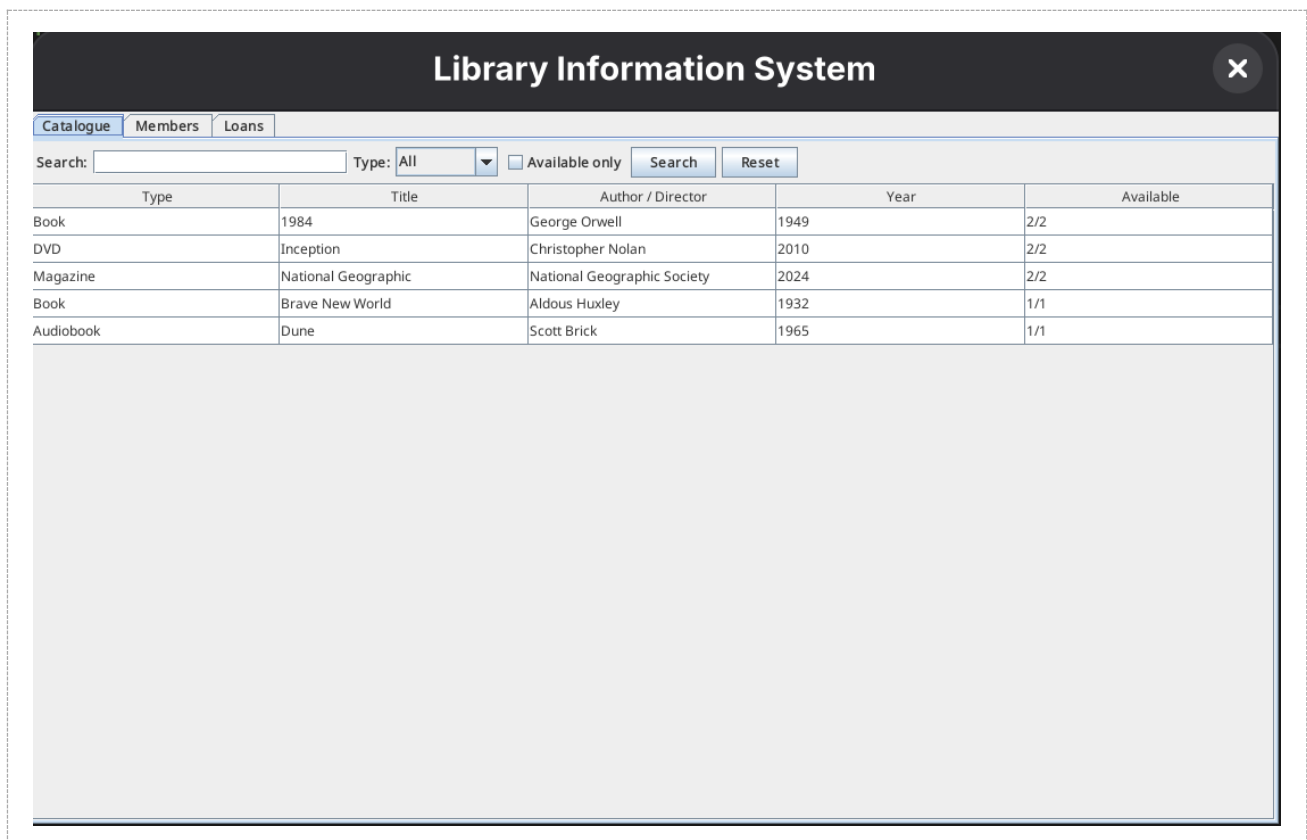


Figure 1. Catalogue tab with the search and filter bar

Library Information System

CatalogueMembersLoans

Alice Martin [MEM001]

Bob Smith [MEM002]

Name : Alice Martin

ID : MEM001

Email : alice@mail.com

Active loans:
No active loans.

Borrowing history:
Loan[L1781273940891] Alice Martin --> "1984" | returned on 2026-06-12

Figure 2. Members tab with a selected member and the borrowing history

Library Information System

CatalogueMembersLoans

Borrow Item

Return Item

Refresh

Member	Item	Borrow Date	Due Date	Status	Fine
Alice Martin	1984	2026-05-23	2026-06-06	OVERDUE 6 day(s)	1,20 EUR
Bob Smith	Brave New World	2026-06-12	2026-06-26	Active	-

Figure 3. Loans tab with an overdue loan highlighted in red

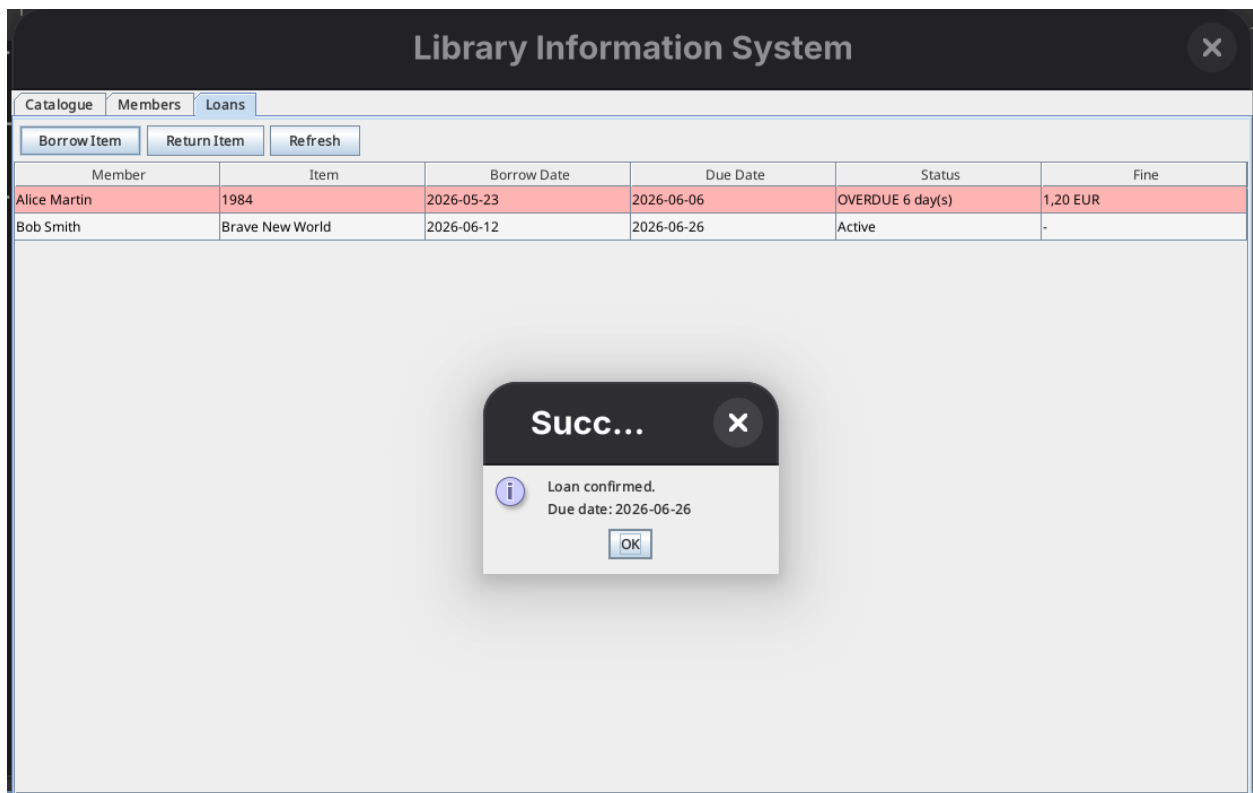
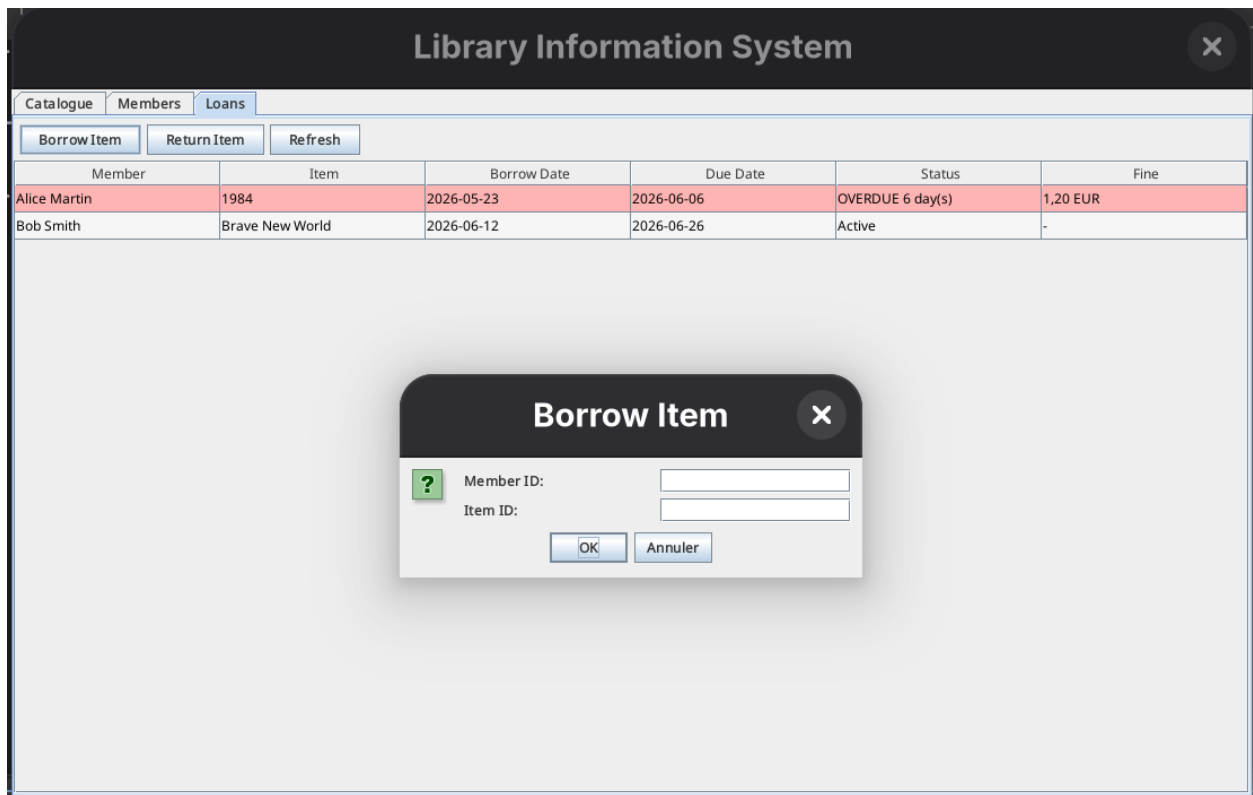


Figure 4. Borrow dialog and the confirmation showing the due date

7. Challenges and solutions

7.1 Decimal separator in the CSV files

The hardest bug I found was that the audiobook durations were written into the CSV file with a comma as the decimal separator, because my computer uses a French locale. A duration of 21.5 hours was written as 21,5, and then the CSV parser split it into two fields, which shifted every following column and broke the line. I fixed it by forcing the US locale in the `String.format` call, so the dot is always used, whatever the machine settings are.

```
return String.format(Locale.US, "AUDIOBOOK,%s,%s,%s,%.1f,%d,%s,%d,%d",
    getItemId(), getTitle(), narrator, durationHours,
    getPublicationYear(), format, getTotalCopies(), getCopiesAvailable());
```

7.2 Keeping the overdue calculation correct after a restart

The overdue status depends on the current date, so it must not be frozen when I save. My solution was to save the borrow date instead of a precomputed overdue flag. When a loan is loaded again, the due date is recomputed from the borrow date, and the overdue status is checked against today's date. So the result is always up to date, even if the application was closed for several days.

7.3 Restoring the exact availability when loading

When an item is rebuilt from the CSV, its constructor first sets `copiesAvailable` equal to `totalCopies`. But if some copies were on loan when the data was saved, the available count has to be set back to its saved value. I wrote a small helper method that sets `copiesAvailable` to the exact number read from the file after the object is built, so an item that was partly borrowed comes back in the right state.

7.4 Separating the GUI from the logic

To make the GUI never touch the internal collections, I had to expose the right high-level methods on the `Library` class. It took a few tries, but it gave a cleaner design in the end, because each panel just calls a method and shows the result.

8. References

- Loïc Yon, My Java course from home university and prior knowledge I had, <https://perso.isima.fr/loic/java/index.php>
- Joyce Farrell, Java Programming, 9th edition, Cengage. Main course textbook.
- Loy, Niemeyer and Leuck, Learning Java, 5th edition, O'Reilly. Used for exceptions, file I/O and collections.
- Oracle, Java SE API documentation, <https://docs.oracle.com/javase/> . Reference for java.time, collections and Swing.
- Oracle, The Java Tutorials, Creating a GUI with Swing. Used for JTable, AbstractTableModel and cell rendering.

Appendix: note on the GitHub commit history

Some commits in the repository show the GitHub username linaxmlk. I made these commits myself from another computer (my girlfriend's laptop) while I was working away from my usual setup. All the work in the repository was written by me.